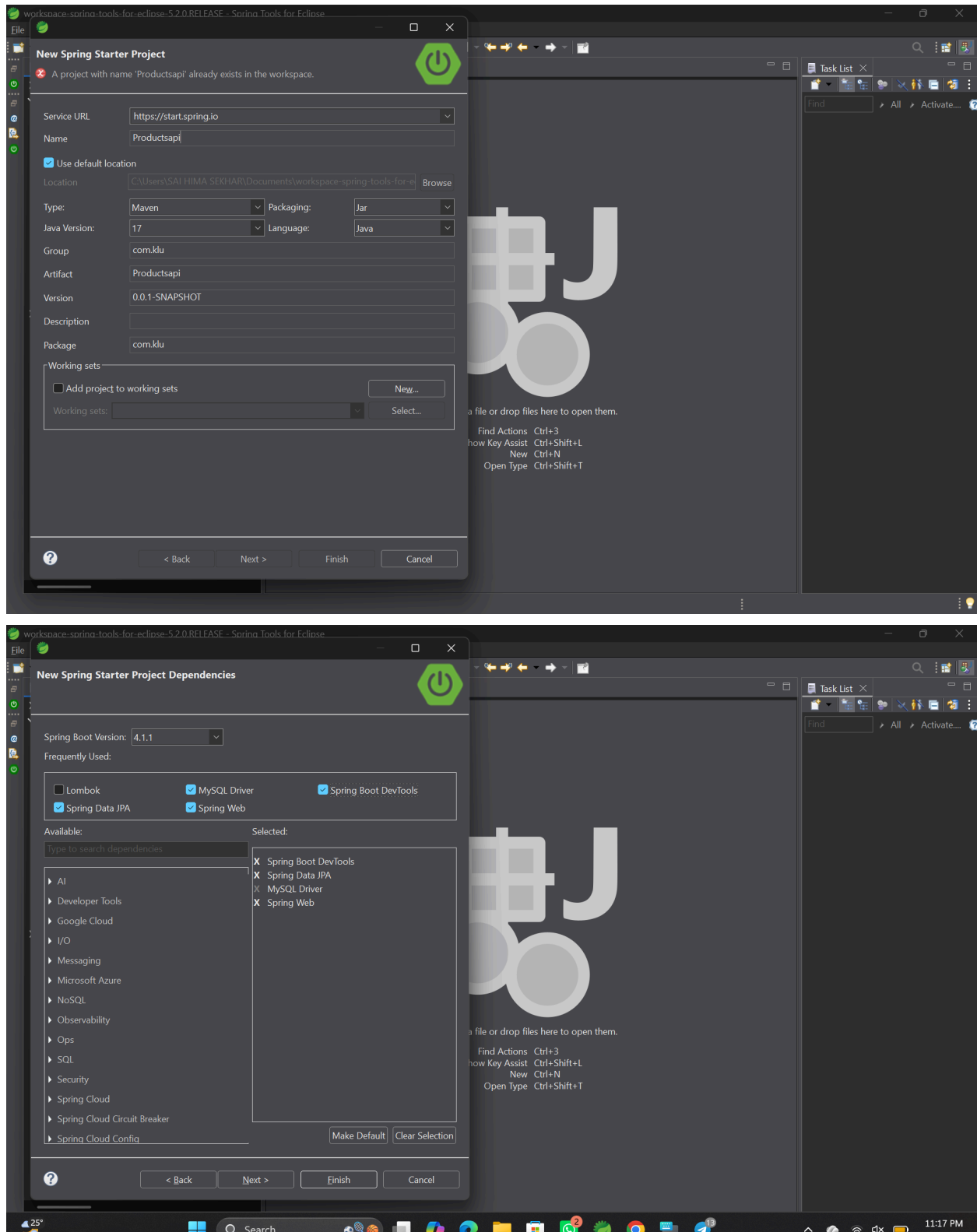
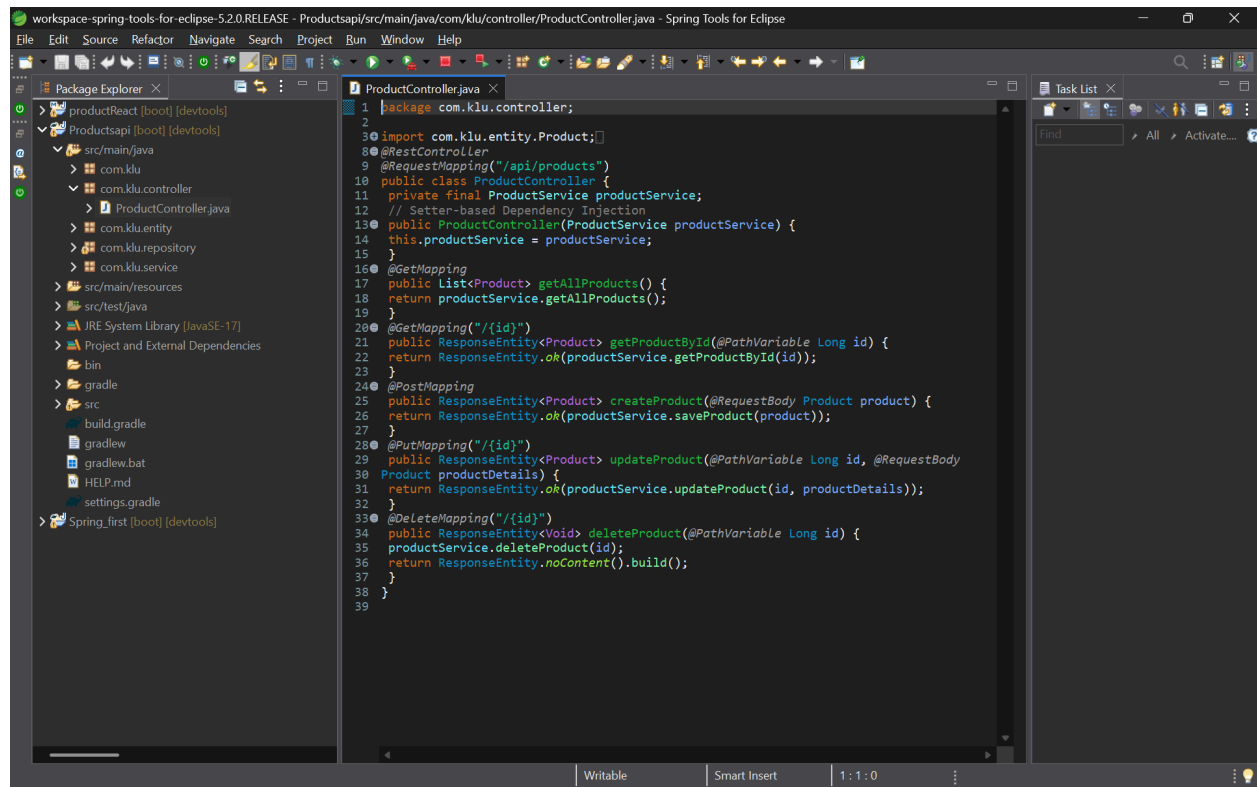


1. Project creation:



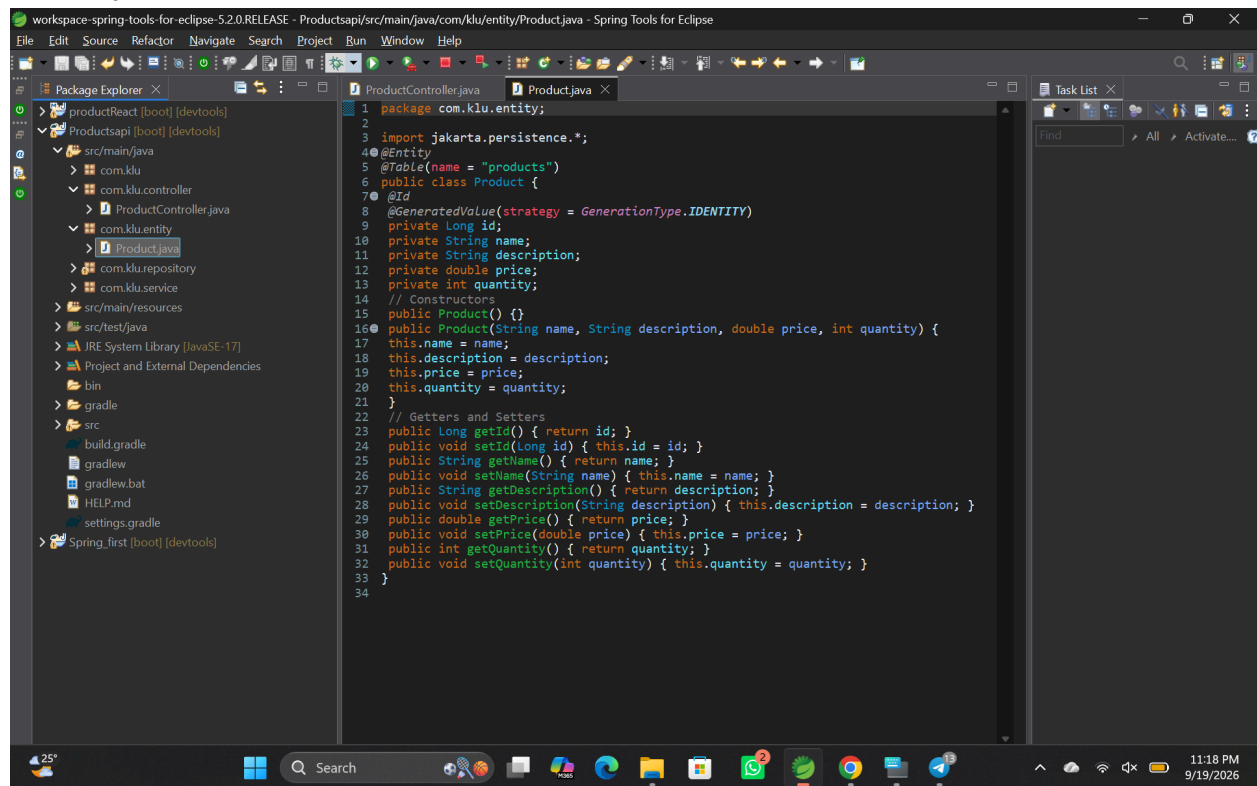
ProductController.java



The screenshot shows the Eclipse IDE with the 'ProductController.java' file open. The Package Explorer on the left shows the project structure: 'productReact [boot] [devtools]' containing 'Productsapi [boot] [devtools]' which has 'src/main/java' containing 'com.klu' containing 'controller' containing 'ProductController.java'. The main editor displays the code for 'ProductController.java'.

```
1 package com.klu.controller;
2
3 import com.klu.entity.Product;
4
5 @RestController
6 @RequestMapping("/api/products")
7 public class ProductController {
8     private final ProductService productService;
9     // Setter-based Dependency Injection
10    public ProductController(ProductService productService) {
11        this.productService = productService;
12    }
13
14    @GetMapping
15    public List<Product> getAllProducts() {
16        return productService.getAllProducts();
17    }
18
19    @GetMapping("/{id}")
20    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
21        return ResponseEntity.ok(productService.getProductById(id));
22    }
23
24    @PostMapping
25    public ResponseEntity<Product> createProduct(@RequestBody Product product) {
26        return ResponseEntity.ok(productService.saveProduct(product));
27    }
28
29    @PutMapping("/{id}")
30    public ResponseEntity<Product> updateProduct(@PathVariable Long id, @RequestBody
31        Product productDetails) {
32        return ResponseEntity.ok(productService.updateProduct(id, productDetails));
33    }
34
35    @DeleteMapping("/{id}")
36    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
37        productService.deleteProduct(id);
38        return ResponseEntity.noContent().build();
39    }
40 }
```

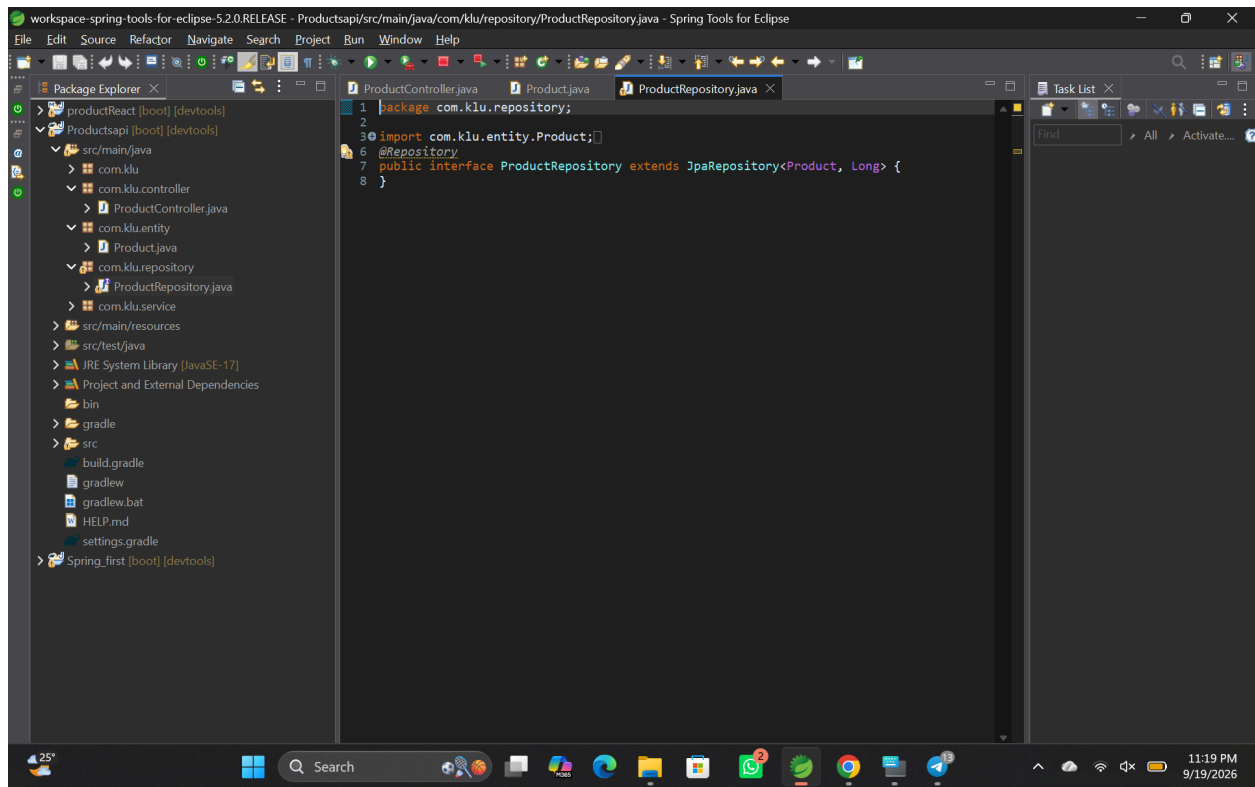
Product.java



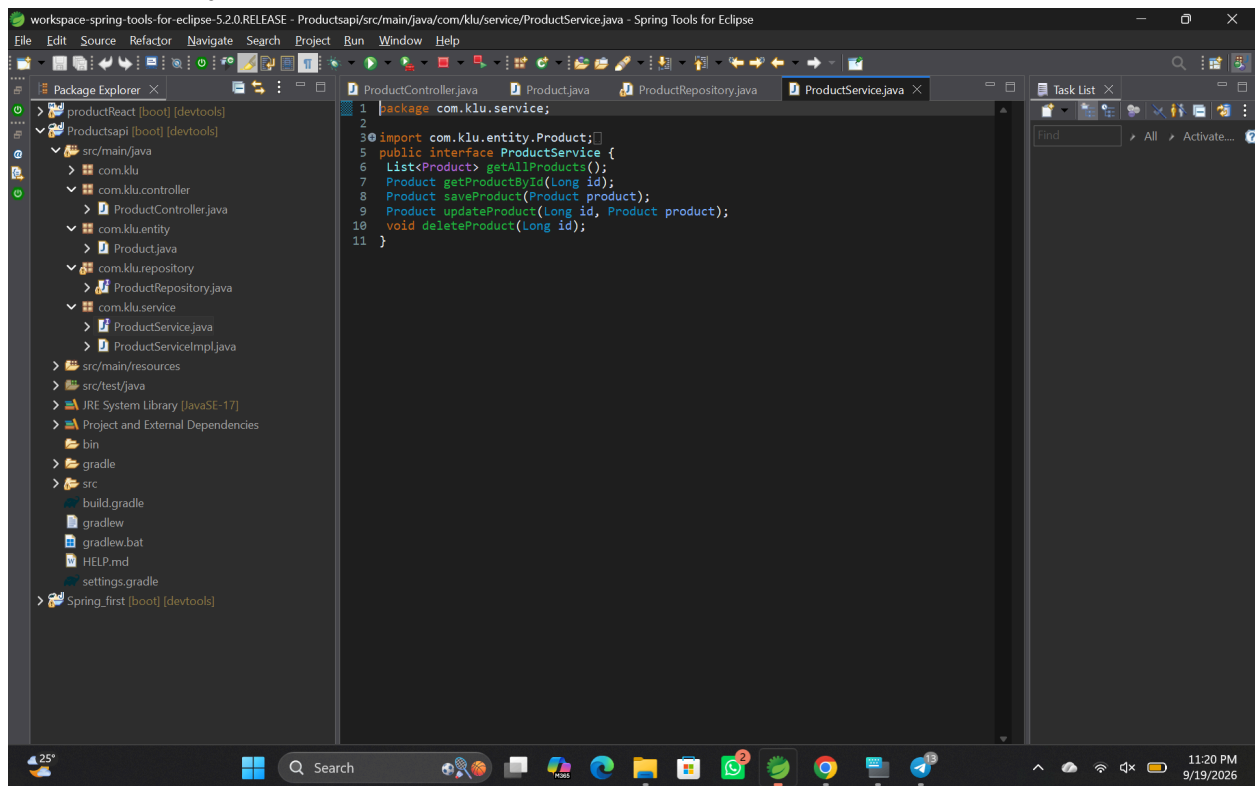
The screenshot shows the Eclipse IDE with the 'Product.java' file open. The Package Explorer on the left shows the project structure: 'productReact [boot] [devtools]' containing 'Productsapi [boot] [devtools]' which has 'src/main/java' containing 'com.klu' containing 'entity' containing 'Product.java'. The main editor displays the code for 'Product.java'.

```
1 package com.klu.entity;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "products")
7 public class Product {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11    private String name;
12    private String description;
13    private double price;
14    private int quantity;
15    // Constructors
16    public Product() {}
17    public Product(String name, String description, double price, int quantity) {
18        this.name = name;
19        this.description = description;
20        this.price = price;
21        this.quantity = quantity;
22    }
23    // Getters and Setters
24    public Long getId() { return id; }
25    public void setId(Long id) { this.id = id; }
26    public String getName() { return name; }
27    public void setName(String name) { this.name = name; }
28    public String getDescription() { return description; }
29    public void setDescription(String description) { this.description = description; }
30    public double getPrice() { return price; }
31    public void setPrice(double price) { this.price = price; }
32    public int getQuantity() { return quantity; }
33    public void setQuantity(int quantity) { this.quantity = quantity; }
34 }
```

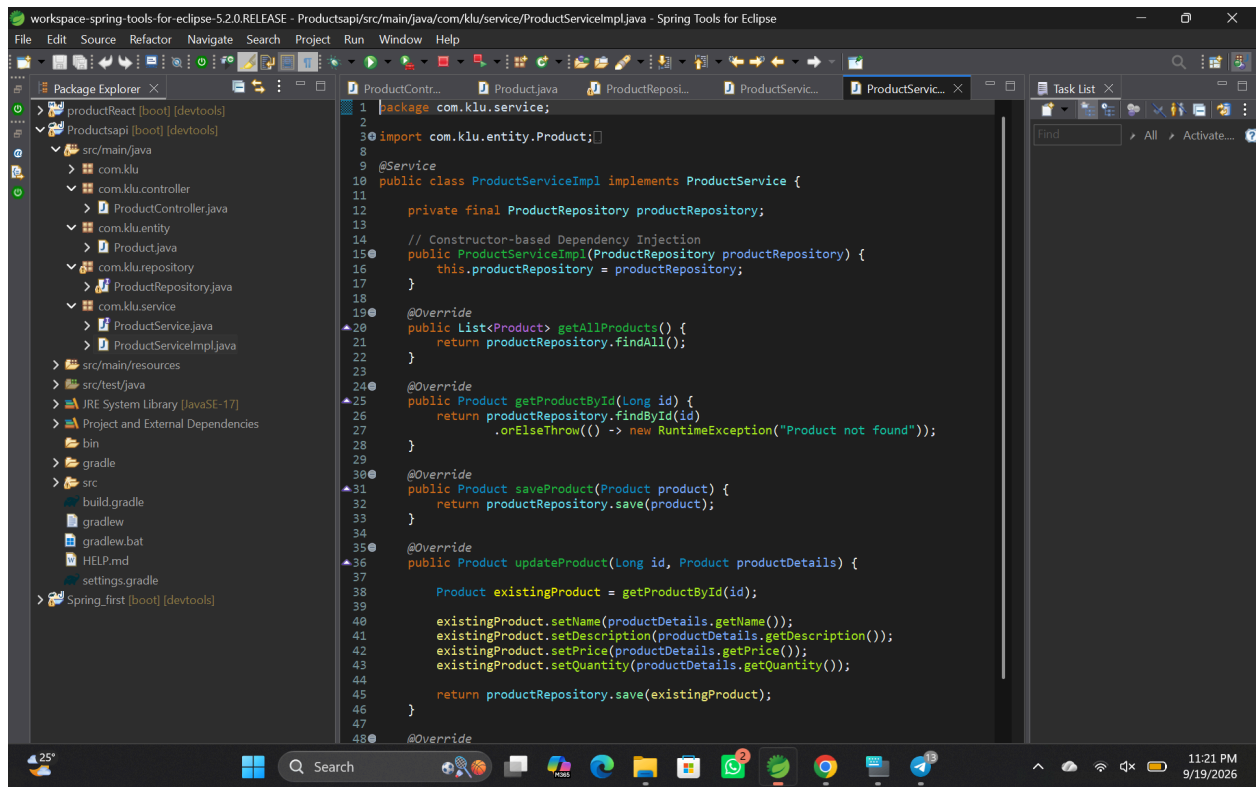
ProductRepository.java



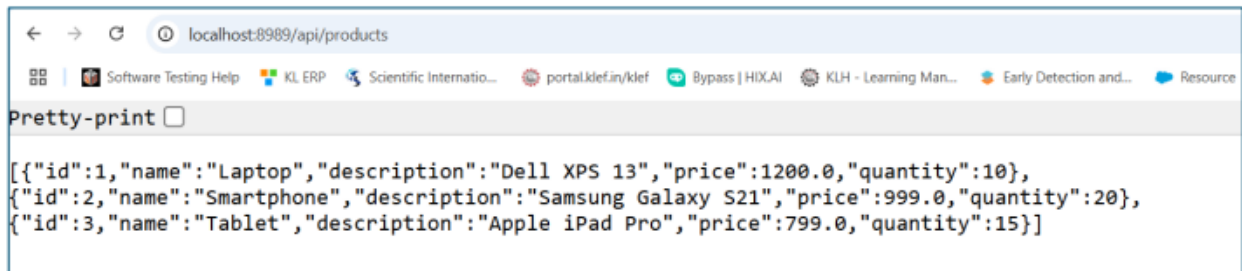
ProductService.java



ProductServiceImpl.java

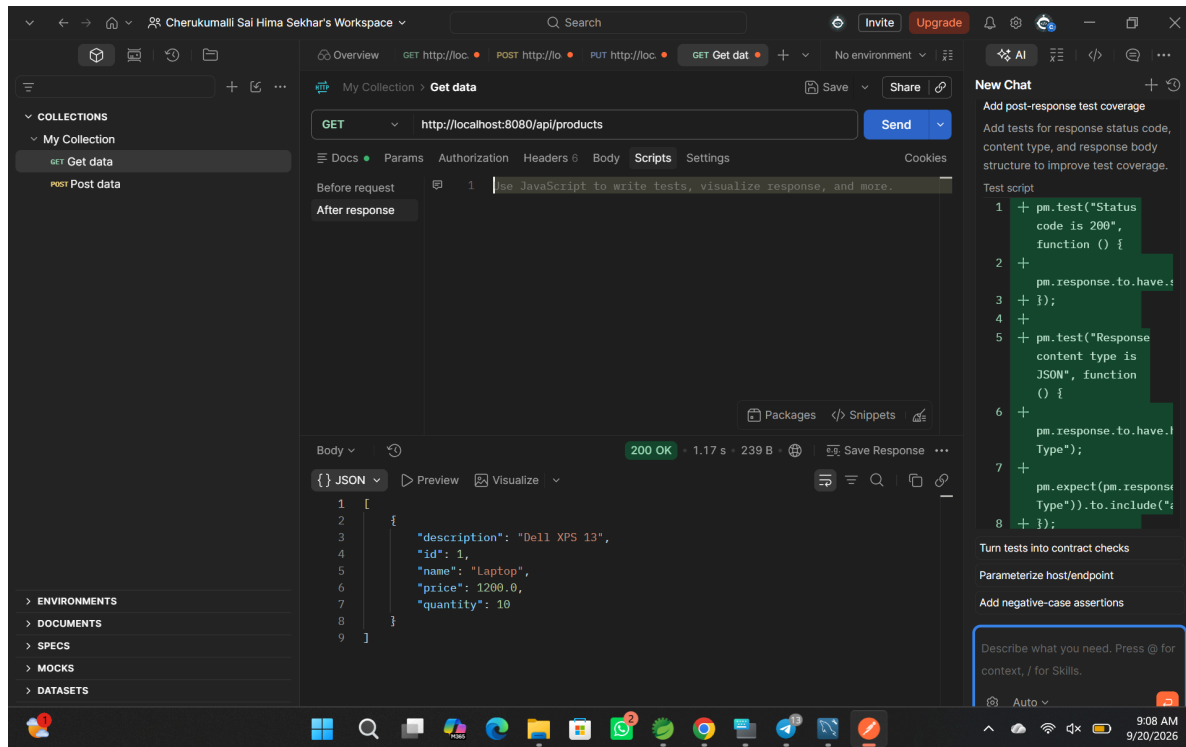


Outputs:

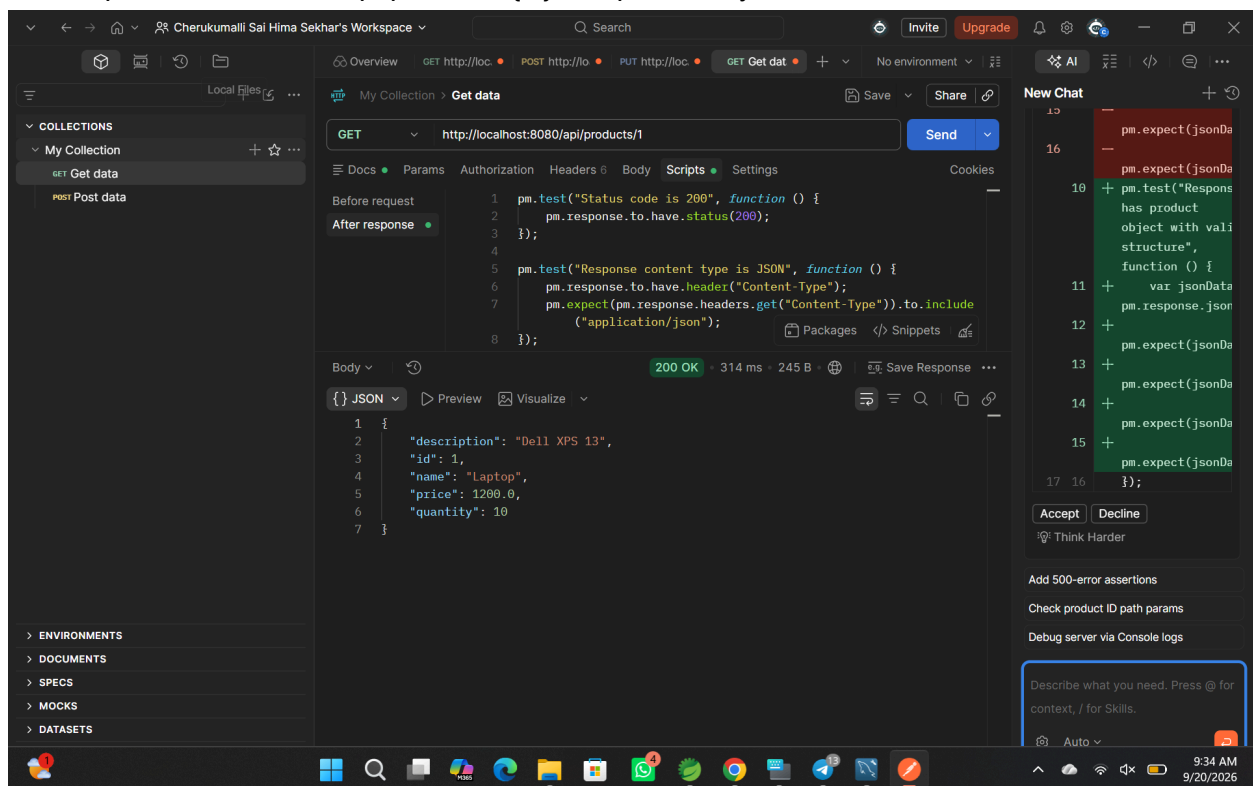


POSTMAN

GET <http://localhost:8080/api/products> Get all products



GET <http://localhost:8989/api/products/{id}> Get product by ID



POST http://localhost:8989/api/products Create new product

The screenshot shows the Postman interface with a POST request to `http://localhost:8080/api/products`. The request body is a JSON object: `{ "name": "Mac", "description": "lcvjnbzkdjuclciubcnc", "price": 1000000000, "quantity": 100 }`. The response is a 200 OK status with a response time of 115 ms and a body of 251 B. The response body is a JSON object: `{ "description": "lcvjnbzkdjuclciubcnc", "id": 2, "name": "Mac", "price": 1.0E8, "quantity": 100 }`. The interface includes a sidebar with collections, environments, documents, specs, mocks, datasets, and flows. A 'New Chat' panel on the right offers options to 'Accept' or 'Decline' a failing status code assertion and provides a prompt to 'Describe what you need. Press @ for context, / for Skills.'

PUT http://localhost:8989/api/products/{id} Update existing product

The screenshot shows the Postman interface with a PUT request to `http://localhost:8080/api/products/2`. The request body is a JSON object: `{ "name": "Mac", "description": "lcvjnbzkdjuclciubcnc", "price": 1000000000, "quantity": 1020 }`. The response is a 200 OK status with a response time of 86 ms and a body of 252 B. The response body is a JSON object: `{ "description": "lcvjnbzkdjuclciubcnc", "id": 2, "name": "Mac", "price": 1.0E8, "quantity": 1020 }`. The interface includes a sidebar with collections, environments, documents, specs, mocks, datasets, and flows. A 'New Chat' panel on the right offers options to 'Accept' or 'Decline' a failing test for server error and provides a prompt to 'Describe what you need. Press @ for context, / for Skills.'

DELETE http://localhost:8989/api/products/{id} Delete a product

Postman interface showing a DELETE request to `http://localhost:8080/api/products/2`. The request body is a JSON object:

```
{  "name": "Mac",  "description": "lcvjnbzkdjuclciubcnc",  "price": 100000000,  "quantity": 10204}
```

The response is a `204 No Content` status. The right sidebar shows a chat window with instructions on how to fix the status code assertion.

Fix the status code assertion
Update the status code assertion to accept a 204 No Content response, matching the expected outcome for this DELETE request.

Test script

```
1  - pm.test("Success: POST request", function () {  
2    - pm.expect(pm.response.status).to.be(201);  
3  
4  + pm.test("Success: DELETE request", function () {  
5    + pm.response.status.to.be(204);  
6  });
```

Accept Decline
Think Harder

Fix the failing PUT tests
Add negative-case assertions
Parameterize product id & body

Describe what you need. Press @ for context, / for Skills.

Auto

MySQL Workbench interface showing a query window with the following SQL commands:

```
1 show databases;  
2 use product;  
3 show tables;  
4 desc products;  
5 select * from products;
```

The query window displays the result grid for the 'products' table:

id	description	name	price	quantity
1	Dell XPS 13	Laptop	1200	10
3	lcvjnbzkdjuclciubcnc	Mac	100000000	1020

The right sidebar shows a chat window with instructions on how to toggle automatic help.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.